

TÀI LIỆU KỸ THUẬT

DỰ ÁN PERSONAL FINANCE MANAGEMENT (FINJAR)

Phiên bản: 1.0

Ngày phát hành: 26/05/2026

Đối tượng đọc: Đội ngũ phát triển (Backend, Frontend, DevOps, QA)

MỤC LỤC

- Giới thiệu dự án
- Kiến trúc tổng quan
- Công nghệ sử dụng (Tech Stack)
- Cấu trúc thư mục dự án
- Mô hình dữ liệu (Database Schema)
- Luồng xử lý chính (Flow Diagrams)
- Tầng API (Controllers & Endpoints)
- Tầng Service (Business Logic)
- Tầng Repository (Data Access)
- Xác thực và phân quyền (Auth)
- Tích hợp bên ngoài (Casso, Gemini AI, OCR)
- Background Jobs
- Middleware & Error Handling
- Quy ước (Conventions)
- Triển khai (Deployment)

1. GIỚI THIỆU DỰ ÁN

FinJar — Personal Finance Management là hệ thống quản lý tài chính cá nhân hướng đến việc **tự động hóa cao** quá trình theo dõi, phân bổ và phân tích chi tiêu của người dùng cá nhân hoặc hộ gia đình.

1.1 Mục tiêu nghiệp vụ chính

- Tự động hóa quản lý chi tiêu** — giảm thao tác nhập liệu thủ công thông qua đồng bộ ngân hàng (Casso) và OCR hóa đơn.
- Hỗ trợ ra quyết định sớm** — cảnh báo vượt hạn mức, nhắc lịch thanh toán, gợi ý từ AI.
- Kết nối chi tiêu với mục tiêu tài chính** — gắn mỗi đồng chi với hũ ngân sách và mục tiêu tiết kiệm cụ thể.

1.2 Đối tượng người dùng

- **End User:** Cá nhân, người nội trợ, người đi làm bận rộn cần công cụ quản lý tự động.
- **Admin:** Vận hành hệ thống, quản lý danh mục mặc định, broadcast, cấu hình AI.

1.3 Phạm vi phiên bản V1

- CRUD đầy đủ: Tài khoản, Hũ, Giao dịch, Danh mục, Hạn mức, Mục tiêu, Nhắc lịch.
- Đồng bộ ngân hàng qua OAuth Casso.
- Import sao kê + OCR ảnh hóa đơn.
- AI chat (Gemini) tư vấn tài chính.
- Dashboard cá nhân & dashboard admin.
- Broadcast hệ thống + Audit log.

2. KIẾN TRÚC TỔNG QUAN

Hệ thống triển khai theo **mô hình N-tier (3-layer)** truyền thống của ASP.NET Core, có tách biệt rõ ràng theo Domain Module.

2.1 Sơ đồ kiến trúc tổng quan



2.2 Nguyên tắc phân lớp

- **Api layer:** chỉ chứa HTTP concerns (routing, validation request, response format). Không chứa logic nghiệp vụ.
- **Service layer:** chứa toàn bộ business rule, validation nghiệp vụ, orchestration. Không trực tiếp expose ra HTTP.
- **Repository layer:** chứa Entity, ApplicationDbContext, Migrations. Hiện tại Service truy cập trực tiếp DbContext (phase 2.5 sẽ tách Repository pattern).

3. CÔNG NGHỆ SỬ DỤNG (TECH STACK)

Hạng mục	Công nghệ	Phiên bản
Runtime	.NET	8.0
Framework	ASP.NET Core	8.0
ORM	Entity Framework Core	8.0.0
Database	PostgreSQL	16
EF Provider	Npgsql.EntityFrameworkCore.PostgreSQL	8.0.0
Naming Convention	EFCore.NamingConventions (snake_case)	8.0.3
Authentication	Microsoft.AspNetCore.Authentication.JwtBearer	8.0.0
JWT	System.IdentityModel.Tokens.Jwt	8.16.0
Password Hashing	BCrypt.Net-Next	4.0.3
Validation	FluentValidation	12.1.1
Logging	Serilog (Console + File)	8.0.3
API Docs	Swashbuckle (Swagger)	6.6.2
Background Jobs	Quartz + HostedService	3.17.1
Email	MailKit (SMTP)	4.15.1
AI	Google.GenAI (Gemini)	1.6.1
OCR	Sdcb.PaddleOCR + PaddleInference	3.0.1
Image	SixLabors.ImageSharp	3.1.1
Cloud Storage	CloudinaryDotNet	1.28.0
Container	Docker + Docker Compose	—

4. CẤU TRÚC THƯ MỤC DỰ ÁN

```
Personal_Finance_App_Be/
├── docs/                                # Tài liệu nội bộ
│   ├── API V2.md                       # Đặc tả endpoint
│   ├── conventions.md                  # Quy ước thống nhất
│   ├── finjar_schema.sql               # Schema PostgreSQL
│   ├── flow.md                         # Mô tả luồng FE/BE
│   ├── note.md                         # Ghi chú tiến độ
│   ├── overview.md                     # Tổng quan sản phẩm
│   └── user_story.md                   # User stories
├── Personal_Finance_Management/
│   ├── Personal_Finance_Management.Api/ # Tầng HTTP
│   │   ├── Controllers/                # 22 controllers
│   │   ├── Middlewares/                # CorrelationId, GlobalEx
│   │   ├── Extensions/                 # Jwt, Authz, Swagger, DB
│   │   ├── Jobs/                       # BroadcastDispatchBg
│   │   ├── Contracts/                  # ApiErrorResponse
│   │   ├── Program.cs
│   │   └── appsettings.json
│   ├── Personal_Finance_Management.Service/ # Tầng nghiệp vụ
│   │   ├── Auth/ JwtService/ User/ Onboarding/
│   │   ├── FinancialAccount/ Jar/ Transaction/
│   │   ├── Category/ Limit/ Goal/ Reminder/
│   │   ├── Dashboard/ Notification/ broadcast/ admin/
│   │   ├── BankConnection/ BankSync/ Casso/
│   │   ├── import/ ocr/ ai/
│   │   ├── Common/                    # Constants, Enums
│   │   ├── Base/                      # PagedResult, CurrentUser
│   │   ├── Validations/                # FluentValidation
│   │   ├── seeding/                   # Seed accounts
│   │   └── baseServices/
│   ├── Personal_Finance_Management.Repository/
│   │   ├── AppDbContext.cs             # 1005 dòng – cấu hình 18 bảng
│   │   ├── Entity/                    # 18 entities
│   │   ├── Enum/                      # 16 enums
│   │   └── Migrations/                 # EF migrations
├── Dockerfile
├── docker-compose.yml
└── render.yaml                         # Cấu hình Render.com
```

4.1 Quy tắc đặt tên trong Service

Mỗi module nghiệp vụ trong **Service/** đều có cấu trúc thống nhất:

```
ModuleName/
├── IService.cs        # interface
├── Service.cs          # implementation
├── Request.cs          # DTO input
└── Response.cs         # DTO output
```

5. MÔ HÌNH DỮ LIỆU (DATABASE SCHEMA)

Cơ sở dữ liệu PostgreSQL 16, 18 bảng, dùng **snake_case**, mọi PK là **UUID**, thời gian là **TIMESTAMPTZ** (UTC).

5.1 Sơ đồ ERD đơn giản hóa

5.2 Mô tả các bảng chính

accounts — Tài khoản người dùng

Cột	Kiểu	Mô tả
id	UUID PK	
role_id	UUID FK	Tham chiếu roles
username, email	VARCHAR UNIQUE	
password_hash	TEXT	BCrypt hash
status	VARCHAR	Active/Banned
preferred_currency	CHAR(3)	DEFAULT 'VND'
is_onboarding_completed	BOOL	
last_login_at	TIMESTAMPTZ	

financial_accounts — Nguồn tiền (ví, ngân hàng)

Cột	Kiểu	Ghi chú
account_type	VARCHAR	Cash / Bank / EWallet / Other
connection_mode	VARCHAR	Manual / LinkedApi
provider_code	VARCHAR	'casso' nếu liên kết
access_token_ref	TEXT	Encrypted token
sync_status	VARCHAR	NeverSynced/Synced/Syncing/Error/Disconnected
current_balance	NUMERIC(18,2)	
is_default	BOOL	

jars — Hũ ngân sách

- `balance` do hệ thống quản lý qua allocate/transaction, **client không bao giờ set trực tiếp**.
- `status`: Active / Paused / Archived.

transactions — Giao dịch

- `type`: Income / Expense (V1 **không có Transfer**).
- `transactions_amount`: lưu **có dấu** (Income > 0, Expense < 0). API luôn trả/nhận giá trị dương.
- `source_type`: Manual / Imported / OCR / Jar / System.
- Soft delete: `is_deleted`, `deleted_at`.
- Indexes filter `WHERE is_deleted = FALSE`.

categories

- Default categories (`is_default = TRUE`, `owner_user_id = NULL`) do Admin tạo.
- Custom categories thuộc user (`owner_user_id = user.id`).

spending_limits

- Constraint: `jar_id IS NOT NULL OR category_id IS NOT NULL` (ít nhất một).
- `period`: Daily / Monthly.
- `alert_at_percentage`: 0–100.

goals & goal_contributions

- Goal có thể `linked_jar_id` để đóng góp từ hũ cụ thể.
- Mỗi contribution **tăng** `goals.saved_amount` và (nếu có sourceJarId) **giảm** `jars.balance`.

reminders

- `frequency`: Daily / Weekly / Monthly / Quarterly / Yearly.
- `notify_days_before`: cảnh báo trước N ngày.

import_jobs & import_transaction_drafts

- Trạng thái: Pending → Processing → AwaitingReview → Completed/Failed.
- Drafts cho phép người dùng review trước khi xác nhận tạo transaction.

broadcasts & notifications

- Broadcast gửi cho `target_audience` (default 'All').
- Khi dispatch sẽ fan-out tạo `notifications` cho từng user.

audit_logs

- Ghi lại mọi action quan trọng của admin: Ban, Unban, CreateCategory, CreateBroadcast...
- Lưu IP, JSON metadata.

bank_connection_sessions

- OAuth PKCE state cho Casso flow.
 - Status: Pending → Authorized → Completed/Failed/Expired.
-

6. LƯỒNG XỬ LÝ CHÍNH (FLOW DIAGRAMS)

6.1 Flow đăng ký & đăng nhập



6.2 Flow ghi nhận giao dịch (Expense)

FE gửi: POST /api/v1/transactions
{ type: "Expense", transactionsAmount: 50000,
financialAccountId, jarId, categoryId, note, transactionDate }

Server:

1. AuthZ: User policy (JWT)
2. TransactionService.Create()
 - a. Validate jar/account/category thuộc currentUser (IDOR check)
 - b. Quyết định dấu amount: Expense → amount * -1
 - c. Insert transaction (source_type = Manual)
 - d. Cập nhật jars.balance += amount (giảm vì amount < 0)
 - e. Cập nhật financial_accounts.current_balance
 - f. Kiểm tra spending_limits liên quan → enqueue LimitAlertJob nếu cần
3. Trả 201 { id, ... } (amount luôn dương theo convention)

6.3 Flow liên kết ngân hàng qua Casso (OAuth)

```
[User] [FE] [BE] [Casso]
| | | | |
| | POST /financial-accounts/casso/connect | |
| | -----> | |
| | | BankConnectionService | |
| | | - tạo state + code_verifier | |
| | | - lưu bank_connection_session | |
| | | redirect URL < | |
| | <----- | |
| | window.location = casso/authorize?state=... | |
| | -----> | |
| | user login & consent [Casso UI] | |
| | <----- redirect /casso/callback?code=...&state=... <---- | |
| | GET /financial-accounts/casso/callback | |
| | -----> | |
| | | CassoClient.ExchangeToken --> | |
| | | <----- access_token ----- | |
| | | Encrypt token (AES) | |
| | | Insert/Update financial_account | |
| | | (connection_mode = LinkedApi) | |
| | <----- redirect FE return_url | |
| | |
| | Sau đó FE gọi: | |
| | POST /financial-accounts/{id}/sync | |
| | -----> | BankSyncService.Sync() | |
| | | - decrypt token | |
| | | - CassoClient.GetTransactions | |
| | | - upsert transactions (dedupe | |
| | | qua external_transaction_id) | |
| | | - cập nhật last_synced_at | |
| | <----- 200 { syncedCount } | |
```

6.4 Flow import / OCR hóa đơn

1. POST /api/v1/imports (multipart file)
 - ImportService tạo import_job (status=Pending)
 - Lưu file vào /uploads
 - Nếu là ảnh: gọi OCR.Service (PaddleOCR)
 - ReceiptParserService trích amount/date/merchant
 - Sinh ra import_transaction_drafts (status=AwaitingReview)
2. GET /api/v1/imports/{id}
 - Trả về danh sách draft để FE preview
3. PATCH /api/v1/imports/{id}/drafts/{draftId}
 - Người dùng sửa: category, jar, note, amount, type
4. POST /api/v1/imports/{id}/confirm
 - Với mỗi draft hợp lệ: tạo transaction (source_type=Imported hoặc OCR)
 - Cập nhật jars, financial_account
 - Cập nhật import_job.status = Completed
5. DELETE /api/v1/imports/{id} → Hủy import job + drafts

6.5 Flow cảnh báo vượt hạn mức

```
Sau mỗi POST/PATCH/DELETE transaction:
  TransactionService → LimitEvaluator.Check(userId, jarId, categoryId)
    → Tính tổng chi trong period (Daily / Monthly)
    → if totalSpent >= limit.amount * alertAtPercentage / 100:
      Insert notification (type=SpendingAlert)

Background: LimitAlertJob (mỗi giờ - phase 4)
  → Quét limit chưa cảnh báo hôm nay → bổ sung notification
```

6.6 Flow broadcast hệ thống

```
Admin tạo broadcast (POST /admin/broadcasts) → status=Queued

BroadcastDispatchBackgroundService (PeriodicTimer N giây):
  → SELECT broadcasts WHERE status=Queued AND scheduled_at <= NOW()
  → Với mỗi user thuộc target_audience:
    INSERT notifications (broadcast_id, type=Broadcast)
  → Cập nhật broadcast.status=Sent, delivered_count, sent_at
```

7. TẦNG API — CONTROLLERS & ENDPOINTS

7.1 Tổng quan controller (22 controllers)

Nhóm	Controllers
Auth & User	AuthController, UserController, OnboardingController
Tài chính	FinancialAccountController, JarController, CategoryController, TransactionsController
Phân tích	DashboardController, LimitController, GoalController, ReminderController
Hệ thống	NotificationController, AIChatController, ImportController, HealthController
Admin	AdminUserController, AdminCategoryController, AdminBroadcastController, AdminDashboardController, AdminAuditLogController, AdminAISettingController, AdminChangeRoleController

7.2 Endpoint chính (trích lược API V2)

Tất cả endpoint dùng prefix `/api/v1/`, JSON camelCase, JWT Bearer.

Auth & User - `POST /auth/register` · `POST /auth/login` · `POST /auth/logout` - `GET /user/me` · `PATCH /user/me` · `GET /user/me/setup` - `POST /onboarding`

Financial Accounts - `GET /financial-accounts` - `POST /financial-accounts/Manual` - `POST /financial-accounts/casso/connect` - `GET /financial-accounts/casso/callback` (anonymous, OAuth redirect) - `POST /financial-accounts/{id}/sync` - `PATCH /financial-accounts/{id}` · `DELETE /financial-accounts/{id}`

Jars / Categories / Transactions - `GET|POST|PATCH|DELETE /jars` + `POST /jars/{id}/allocate` - `GET|POST|PATCH|DELETE /categories` - `GET /transactions` (filter: pageIndex, pageSize, financialAccountId, type,

jarId, categoryId, fromDate, toDate, keyword, sortBy) - `POST|PATCH|DELETE /transactions/{id}` - `POST /transactions/Casso` (Casso webhook — Anonymous)

Import / OCR - `POST /imports` · `GET /imports` · `GET /imports/{id}` - `PATCH /imports/{id}/drafts/{draftId}` - `POST /imports/{id}/confirm` · `DELETE /imports/{id}`

Limits / Goals / Reminders / Notifications - `GET|POST|PATCH|DELETE /limits` - `GET|POST|PATCH|DELETE /goals` + `POST /goals/{id}/contributions` - `GET|POST|PATCH|DELETE /reminders` - `GET /notifications` · `PATCH /notifications/status`

Dashboard / AI - `GET /dashboard` - `POST /ai/chat`

Admin (Bearer Admin) - `GET /admin/users`, `PATCH /admin/users/{id}/status`, `PATCH /change-role/{accountId}` - `GET|POST|PATCH|DELETE /admin/categories` - `POST|GET /admin/broadcasts` - `GET /admin/dashboard`, `GET /admin/audit-logs` - `GET|PATCH /admin/ai-settings`

Health - `GET /health`, `/health/db/local`, `/health/db/render`

7.3 Định dạng response chuẩn

Success

```
{ "data": { ... } }
```

Paged

```
{ "items": [...], "totalCount": 120, "page": 1, "pageSize": 20 }
```

Error envelope (4xx/5xx)

```
{
  "code": "VALIDATION_FAILED",
  "message": "amount must be greater than 0",
  "field": "transactionsAmount",
  "details": {},
  "traceId": "00-abc..."
}
```

8. TẦNG SERVICE — BUSINESS LOGIC

8.1 Danh sách module (25+)

Module	Vai trò
Auth	Đăng ký, đăng nhập, hash mật khẩu (BCrypt), kiểm tra status
JwtService	Sinh & validate JWT, mã hóa claims
User	CRUD profile, cập nhật avatar
Onboarding	Wizard khảo sát, gợi ý budget method
FinancialAccount	CRUD nguồn tiền, default account, sync status
BankConnection	Quản lý OAuth session Casso
BankSync	Pull giao dịch từ Casso, dedupe
Casso	HTTP client + AES token protector
Jar	CRUD hũ, allocate balance
Category	CRUD danh mục (user & admin)
Transaction	CRUD giao dịch, sign convention, cập nhật balance
Limit	Spending limit + cảnh báo
Goal	Mục tiêu tiết kiệm + contribution
Reminder	Lịch nhắc thanh toán định kỳ
Dashboard	Aggregate cho user & admin
Notification	Đọc/đánh dấu, lọc, phân trang
Broadcast	Tạo, fan-out, lên lịch
Admin	Ban/unban, change role, audit logs
Import	Sao kê CSV/Excel + OCR pipeline
OCR	PaddleOCR + ReceiptParser
AI	Gemini API client, system prompt FinJar
Validations	Custom FluentValidation + AppValidationException
Common	Constants, Enums, Helpers, PagedResult

8.2 Cross-cutting concerns

- `ICurrentUserAccessor` : lấy `userId` , `role` từ `HttpContext`.
 - `PagedResult<T>` + `QueryableExtensions.ToPagedAsync()` .
 - `AppValidationException` : chuyển thành error envelope ở middleware.
 - `ServiceClaimHelper` / `ServiceTextHelper` : tiện ích chung.
-

9. TẦNG REPOSITORY — DATA ACCESS

9.1 AppDbContext.cs

- File chính: `Personal_Finance_Management.Repository/AppDbContext.cs` (~1005 dòng).
- Cấu hình 18 `DbSet<>`.
- Áp dụng:
- `UseSnakeCaseNamingConvention()` cho cột & bảng.
- CHECK constraints theo enums.
- UNIQUE: `(user_id) WHERE is_default = TRUE` cho `financial_accounts`.
- INDEXES tối ưu theo flow đọc.
- Cascade rule: `Restrict` mặc định, soft delete cho `transactions`, `categories`.

9.2 Migrations

File	Mô tả
20260514094700_Initial	Schema khởi tạo
20260516082910_AddBankConnectionSessions	Bảng OAuth Casso
20260516083634_init v2	Tinh chỉnh schema
20260516123700_AddFinancialAccountDefaultConstraint	Unique default account

`ApplyMigrations=true` → tự `Database.Migrate()` khi khởi động.

10. XÁC THỰC & PHÂN QUYỀN (AUTH)

10.1 JWT Configuration

```
"Jwt": {
  "SecretKey": "<min 32 chars>",
  "Issuer": "PersonalFinanceManagement",
  "Audience": "PersonalFinanceManagement",
  "ExpireMinutes": 120
}
```

- Algorithm: **HMAC SHA-256**.
- Token validation: issuer, audience, signing key, lifetime.
- `RequireHttpsMetadata = false` (development).

10.2 Authorization Policies

- **Policy User**: yêu cầu authenticated user có role `User`.
- **Policy Admin**: yêu cầu authenticated user có role `Admin`.
- Mapping qua `Account.RoleId → Role.Code`.

10.3 Bảo vệ IDOR

Mọi truy vấn dữ liệu **phải** kèm `userId` lấy từ `ICurrentUserAccessor.UserId`. Service không bao giờ trust id từ route nếu chưa cross-check.

11. TÍCH HỢP BÊN NGOÀI

11.1 Casso (OAuth + Sync giao dịch)

Cấu hình `.env`:

```
Casso__BaseUrl=https://oauth.casso.vn/v2
Casso__AuthorizationUrl=https://oauth.casso.vn/auth/authorize
Casso__TokenUrl=https://oauth.casso.vn/auth/token
Casso__ClientId=...
Casso__ClientSecret=...
Casso__RedirectUri=http://localhost:5284/api/v1/financial-accounts/casso/callback
Casso__TokenEncryptionKey=...
Casso__WebhookSecureToken=...
Casso__TimeoutSeconds=30
```

- `CassoClient`: HTTP client (15.4KB) — OAuth code exchange, fetch transactions.
- `CassoTokenProtector`: AES encrypt/decrypt access token trước khi lưu DB.
- Webhook: `POST /transactions/Casso` (Anonymous, verify `WebhookSecureToken`).

11.2 Google Gemini (AI Chat)

```
GoogleAI__ApiKey=...
GoogleAI__DefaultModel=gemini-2.5-flash
GoogleAI__Temperature=0.7
GoogleAI__MaxTokens=1000
GoogleAI__SystemPrompt=You are FinJar, a personal finance assistant...
```

- Module: `Service/ai/Service.cs` (~17.6KB).
- Admin có thể cấu hình lại qua `PATCH /admin/ai-settings`.

11.3 OCR (PaddleOCR)

- `Sdcb.PaddleOCR` + `PaddleInference` 3.0.1.
- Pipeline: upload ảnh → preprocess (`SixLabors.ImageSharp`) → PaddleOCR detect/recognize → `ReceiptParserService` regex-extract amount/date/merchant.

11.4 Email (Gmail SMTP) & Cloudinary

- MailKit SMTP cho thông báo email & reset mật khẩu.
 - Cloudinary lưu avatar & ảnh hóa đơn.
-

12. BACKGROUND JOBS

Job	Cơ chế	Tần suất	Trạng thái
BroadcastDispatchBackgroundService	BackgroundService + PeriodicTimer	BroadcastDispatch.IntervalSeconds	Đã có (Api/Jobs/)
ReminderDispatcherJob	Quartz	5 phút	Phase 4 — đang triển khai
LimitAlertJob	Quartz	Mỗi giờ + trigger sau transaction	Phase 4
ImportJobProcessor	Quartz	On-demand	Phase 4

BroadcastDispatchBackgroundService dùng IServiceScopeFactory để inject DbContext theo scope mỗi vòng lặp.

13. MIDDLEWARE & ERROR HANDLING

13.1 Pipeline trong Program.cs

```
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseMiddleware<GlobalExceptionHandlerMiddleware>();
app.UseSerilogRequestLogging();
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

13.2 CorrelationIdMiddleware

- Header: X-Correlation-Id (sinh nếu thiếu).
- Push vào Serilog LogContext để mọi log có cùng correlation id.

13.3 GlobalExceptionHandlerMiddleware

- Bắt mọi exception.
- AppValidationException → 422 + envelope chuẩn.
- Khác → 500 + sanitized message + log full stack.

13.4 Logging (Serilog)

- Console template: [{Timestamp:HH:mm:ss} {Level:u3}] {CorrelationId} {UserId} {SourceContext} {Message}.
- File: logs/app-.log, daily rolling, giữ 14 ngày.

14. QUY ƯỚC (CONVENTIONS)

14.1 URL & JSON

- Prefix `/api/v1/...`, kebab-case danh từ số nhiều.
- JSON camelCase. `id` cho PK, `<entity>Id` cho FK.
- Boolean: `is*` / `has*` / `can*`.

14.2 Sign convention

- API luôn nhận/trả `amount > 0`.
- Server quyết định dấu theo `type` (Income > 0, Expense < 0).
- Có CHECK constraint DB.

14.3 Pagination & Filter

- `?page=1&pageSize=20` (max 100).
- Response: `{ items, totalCount, page, pageSize }`.
- Sort: `?sort=field` / `?sort=-field`.

14.4 HTTP status

Trường hợp	Status
GET OK	200
POST tạo	201
PATCH OK	200
DELETE OK	204
Validation lỗi	422
Unauthorized	401
Forbidden	403
Not Found	404
Conflict	409
Server error	500

14.5 Timezone

- DB lưu UTC (`timestampz`).
- API trả ISO-8601 UTC (`...Z`).
- FE convert sang `Asia/Ho_Chi_Minh` để hiển thị.

14.6 Soft delete

- Trường `is_deleted` / `deleted_at`.

- Mặc định ẩn trong list/get. Admin có flag `includeDeleted=true`.

14.7 Idempotency

- Import: header `Idempotency-Key`.
- Casso: dedupe theo `external_transaction_id`.

15. TRIỂN KHAI (DEPLOYMENT)

15.1 Dockerfile (multi-stage)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS restore
... restore + build
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
ENV ASPNETCORE_ENVIRONMENT=Production
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "Personal_Finance_Management.Api.dll"]
```

15.2 docker-compose.yml

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: PersonalFinanceManagementDb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: *****
    ports: ["5432:5432"]
    healthcheck: pg_isready
  api:
    build: .
    depends_on: [postgres]
    ports: ["5284:8080"]
    environment:
      ApplyMigrations: "true"
      EnableSwagger: "true"
      ConnectionStrings__DefaultConnection: "Host=postgres;..."
      Jwt__SecretKey: ...
      Casso__: ...
      GoogleAI__: ...
```

15.3 Render.com

- File `render.yaml` cấu hình service production.
- Health check qua `/health`.

15.4 Migrations & seeding

- `ApplyMigrations=true` → tự `Database.Migrate()` khi boot.
- `SeedAccounts`: tạo Admin mặc định nếu DB rỗng.

PHỤ LỤC A — ENUM CHÍNH

Enum	Giá trị
Account.status	Active, Banned
BudgetMethod	SixJars, Rule503020, Custom, Undecided
FinancialAccount.accountType	Cash, Bank, EWallet, Other
FinancialAccount.connectionMode	Manual, LinkedApi
FinancialAccount.syncStatus	NeverSynced, Synced, Syncing, Error, Disconnected
Jar.status	Active, Paused, Archived
Transaction.type	Income, Expense
Transaction.sourceType	Manual, Imported, OCR, Jar, System
ImportJob.status	Pending, Processing, AwaitingReview, Completed, Failed
SpendingLimit.period	Daily, Monthly
Goal.status	Active, Completed, Cancelled
Reminder.frequency	Daily, Weekly, Monthly, Quarterly, Yearly
Reminder.status	Active, Paused, Completed, Cancelled
Broadcast.status	Queued, Sent, Failed, Cancelled
Notification.type	SpendingAlert, GoalUpdate, Reminder, System, Broadcast

PHỤ LỤC B — CÁC ĐIỂM CẦN LƯU Ý KHI PHÁT TRIỂN

1. **Không bao giờ để FE set jars.balance trực tiếp.** Mọi thay đổi đi qua endpoint allocate hoặc transaction.
2. **Mọi service phải lọc theo userId** từ JWT để chống IDOR.
3. **Transfer V1 không tồn tại** — đừng thêm code Transfer chưa thống nhất, dùng `allocate` cho di chuyển tiền giữa hũ.
4. **Soft delete mặc định ẩn** — query phải có `WHERE is_deleted = FALSE`.
5. **Idempotency-Key** bắt buộc cho import & các tác vụ tạo bulk.
6. **Casso token phải được encrypt (AES)** trước khi ghi DB qua `CassoTokenProtector`.
7. **Khi đổi enum**, phải đồng thời update: DB CHECK constraint, file Enum C#, file `Common/Constants`, tài liệu `conventions.md`.
8. **Logging**: dùng Serilog với structured log, không `Console.WriteLine`.
9. **Validation**: dùng FluentValidation + `AppValidationException`, đừng throw `Exception` thô.
10. **Background jobs Phase 4** (Reminder/Limit/Broadcast/Import) phải tận dụng `IServiceScopeFactory` để tránh DbContext bị share giữa các thread.

Hết tài liệu kỹ thuật.